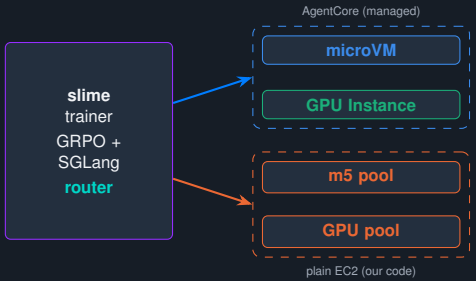
Agentic RL Sandboxes on AWS

A sandbox per episode, at RL scale, with Amazon Bedrock AgentCore and EC2

Kaige Yang
Senior Applied Scientist, AWS

At a glance: one agentic-RL stack on AWS, built for the hard cases

Architecture



Each task runs where its own reference solution passes; the harness and grader are the same everywhere.

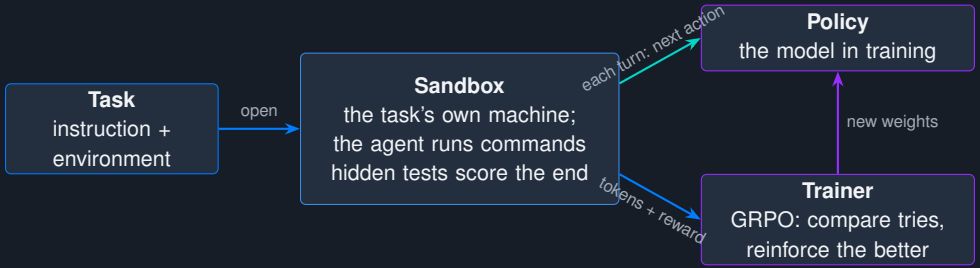
Key contributions

- **A hybrid sandbox tier**: 2 AgentCore + 2 EC2 substrates behind one API, in one training batch.
- **Measured routing**: each task goes where it builds and its own solution passes.
- **Pre-open + heartbeat + warm GPU pool**: sandboxes are ready before the episode starts.
- **slime unpatched**: we plug in through its extension points.

Challenges solved

- **Environments**: **212**/241 Terminal-Bench, **261**/270 KernelBench tasks.
- **Speed**: **1,188** episodes/h; median sandbox wait **0 s**.
- **Rewards**: hidden tests, oracle-checked tasks, speed gate against hacks.
- **Operations**: **0.35 %** infra errors; private network; nothing left running.

Agentic RL: the model learns by acting in a real machine



- **Every try needs its own clean machine**: the agent installs, edits and breaks things.
- **Thousands of tries per hour**: here 1,188 episodes/h, 32 at once.
- **The score must be honest**: RL optimizes the reward, so any shortcut gets learned.
- **The sandbox is the RL environment**: if it's slow, wrong or leaky, training is too.

Amazon Bedrock AgentCore Runtime: a managed machine per session

microVM (the default)

- One session = one isolated microVM with its own kernel; a new session id = a new machine.
- Fully managed: boot, isolation, idle reclaim, teardown.
- Opens in **0.9 s** (median), from your container image.

GPU Instance (capacity provider)

- Same API, a whole GPU machine per session (g5/g6e here).
- Opens in 143 s: a fresh machine every time.

Table 1 What it doesn't do, so we add plain EC2

Limit	Workaround
no Docker daemon, no root capabilities	EC2 **m5** pool
images capped at 2 GB	EC2 (no cap)
microVMs are arm64 only	EC2 x86
1 session = 1 machine, no warm reuse on GPU	EC2 **GPU pool**

In plain English: AgentCore gives every episode its own managed machine in under a second. Where a task needs Docker, root, a big image or warm GPUs, we fill the gap with plain EC2 behind the same API.

Deep dive: "AgentCore Runtime foundations", "What AgentCore can't do"

The workload: 511 real agentic tasks, two very different kinds

Terminal-Bench 241 tasks

Fix or build something in a Linux machine, with shell commands.

Example `postgres-csv-clean`: "clean the customer data in PostgreSQL, export `cleaned_customers.csv`".

Machine: an app container + a Postgres sidecar.

Scored: hidden tests check the CSV and that the work ran in Postgres.

KernelBench 270 tasks

Rewrite a PyTorch model as a faster GPU kernel.

Example: "here is `Model`; write `ModelNew`: same output, on the GPU, faster".

Machine: a GPU with CUDA PyTorch.

Scored: a hidden checker compares outputs and times both: full reward only if $\geq 1.10\times$ faster.

In plain English: every task is a small broken computer plus a hidden exam. Each needs its own image, some need several containers, root or a GPU.

Deep dive: "The agentic tasks in this solution", "One example from each dataset"

16 challenges every agentic-RL sandbox must solve, in four groups

1 Every task gets its environment

one image per task (241 Dockerfiles) · images up to 6.7 GB vs a 2 GB cap · x86-only tasks on arm64 · multi-container tasks · root and capabilities · GPUs

2 The trainer never waits

cold starts of 1 s to 2.5 min vs a ∼11 s agent loop · idle sessions reclaimed after 15 min · concurrency and quotas · long or hung tool calls

3 Rewards you can trust

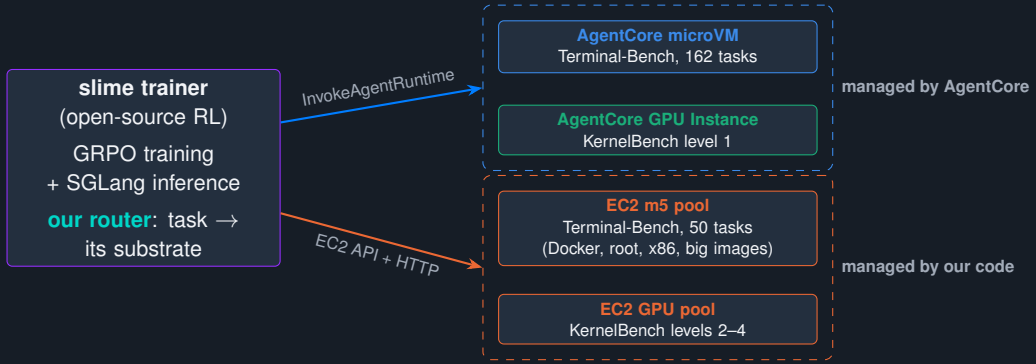
no leaks between episodes · hidden tests the agent can't read or game · tasks whose own solution fails · all-or-nothing scores give GRPO no signal

4 Safe, reliable, affordable

the agent runs untrusted code yet must reach the trainer, with no public ports · nothing left running after a crash · GPU machines dominate the bill

In plain English: the next four slides take one group each: the challenge, what we did, and the number that shows it works.

The solution: one RL loop, four sandbox substrates, chosen per task



One code path: harness, grader and reward never know the substrate.

Routing is data: each task goes where its own reference solution passes.

slime unpatched: we plug in through its extension points.

Deep dive: “The architecture”, “Why four substrates”

Why a hybrid: each substrate covers what the others can't

AgentCore microVM

Terminal-Bench, 162 tasks

Why: fully managed, own kernel per session, **0.9 s** to open.

Can't: arm64 only; 2 GB image cap; no Docker or root; no GPU.

EC2 m5 pool

Terminal-Bench, 50 tasks

Why: a whole VM with Docker, as Terminal-Bench runs upstream: x86, any image size, root, sidecars.

Why not AgentCore: a CPU Instance has the same limits as a microVM.

Price: we own launch, reset and clean-up.

AgentCore GPU Instance

KernelBench level 1, 95 tasks

Why: a small image that needs only a GPU; same API as a microVM, managed, a fresh machine per session.

Can't: microVMs have no GPU; **143 s** to open, as CUDA torch can't fit the 2 GB cap.

EC2 GPU pool

KernelBench levels 2–4, 166 tasks

Why: torch baked in, warm GPUs, a fresh container per episode: **1.4 s** to open.

Why not AgentCore: level 4 needs a 17 GB image; 166 tasks at 143 s each.

Price: we own the lifecycle and verified reset.

Without the hybrid: microVMs alone cover 162 of 241 Terminal-Bench tasks and none of KernelBench. With all four: **212** and **261** of 270, behind one API, in one training batch.

In plain English: use AgentCore wherever a task fits its contract; use plain EC2 when a task needs a whole machine with Docker and root, or a big image with a fast GPU open. The router decides per task, by measurement.

Deep dive: "Why four substrates", "What AgentCore can't do"

Challenge 1: every task gets its environment

What we did

- **Route each task by measurement**: its build, its size, and whether its reference solution passes there.
- **microVM first**; Docker, root, x86 or big images go to an EC2 **m5**.
- **GPUs**: an AgentCore GPU Instance, or a warm EC2 GPU pool with torch baked in.
- **Drop only what can't pass its own check**, with the cause recorded.

Table 2 Coverage (13 Oct)

	used	of
Terminal-Bench	**212**	241
on a **microVM** / on an **m5**	162 / 50	
KernelBench	**261**	270
AgentCore GPU / EC2 GPU pool	95 / 166	

Dropped: 29 Terminal-Bench (rotted upstream, live websites, too slow, never build: none is our harness) and 9 KernelBench (the reference itself needs a 48 GB GPU or a longer grade).

In plain English: a microVM covers two thirds of Terminal-Bench; adding EC2 for the rest and GPUs for KernelBench brings coverage to 88 % and 97 %.

Deep dive: "Coverage" (TB and KB), "What AgentCore can't do"

Challenge 2: the trainer never waits for a sandbox

What we did

- **Pre-open**: look ahead at the next batches and open their sandboxes while this batch runs.
- **Heartbeat**: keep pre-opened sessions alive past the 15 min idle reclaim.
- **Warm GPU pool**: reuse GPU machines, a fresh container per episode.
- **Caps per substrate** so pre-open never exceeds quotas.

Table 3 Full-epoch run: open time vs what the episode waited (median)

	open	waited
AgentCore microVM	0.9 s	0 s
EC2 m5 pool	37 s	0 s
AgentCore GPU Instance	143 s	0 s
EC2 GPU pool (warm)	1.4 s	0 s

1,188 episodes/h, 32 at once, up to 192 live sandboxes. The trainer's wait is the model generating, not the sandbox.

In plain English: opening a sandbox takes from a second to over two minutes, but it happens ahead of time, so the episode starts immediately.

Deep dive: "Sandbox latency", "Time breakdown", "Concurrency"

Challenge 3: rewards you can trust

What we did

- **Hidden tests** are copied in only after the agent stops, under a random name, with a nonce'd result line.
- **Oracle check**: a task trains only where its own reference solution passes.
- **A fresh machine per episode** (AgentCore), or a **verified reset** before reuse (EC2).
- **Partial credit** in training, so near-misses give GRPO a signal.

Table 4 **The reward hack RL found, and the fix**

KernelBench, correctness only: copy `Model` to the GPU	scored 1 in **64 %**
With a 1.10× speed gate	9.5 % of correct kernels

0 dirty resets on reused machines. A grader bug we found while validating was fixed before training.

In plain English: every reward comes from a task whose own solution passes, tests the agent never sees, and a clean machine.

Deep dive: "Reward integrity", "Lifecycles"

Challenge 4: safe, reliable, affordable

Safe

VPC-mode Runtimes, private subnets, **no public ports**; security-group-to-group rules only.

One narrow IAM role per actor: a sandbox can only pull its own image.

Reliable

0.35 % infra errors: 69 in 19,606 episodes over 21 runs.

Every exit path cleans up: stop sessions, terminate VMs, a 30 min dead-man switch on each VM.

Affordable

GPU sandboxes drive the bill: an AgentCore GPU machine lives 16 min for a 5.4 min session.

The warm EC2 GPU pool reuses each machine for ∼9 episodes.

In plain English: the agent runs untrusted code, so nothing is public and nothing outlives the run; GPU reuse is the main cost lever.

Deep dive: “Why security matters”, “What we do for each risk”, “Reliability”, “Cost”

Results: the sandbox side is proven; learning is the next step

Proven (full-epoch run, 13 Oct)

- 4 substrates in one training batch; 212 + 261 tasks.
- 1,188 episodes/h; median sandbox wait 0 s.
- 0.35 % infra errors; honest, gated rewards.

Open

- **Learning signal**: 51 % of GRPO groups have identical rewards, so no gradient.
- Held-out eval is flat at 4B and 9B so far; the 9B solves 5 of 31 held-out microVM tasks.

Next

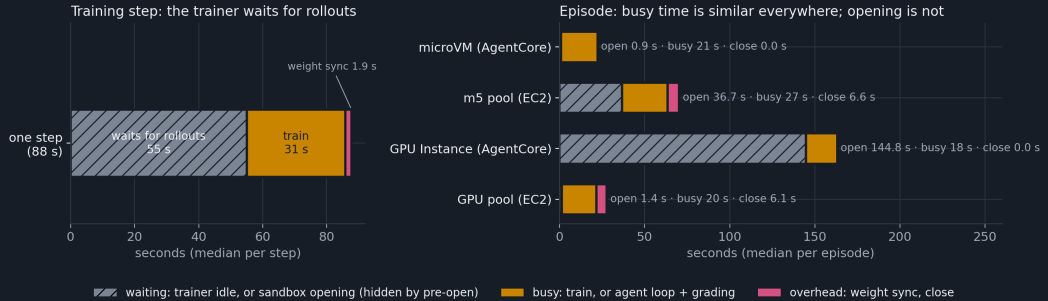
- **Reward shape and task mix**: graded speed reward, a task filter, more tries per prompt, larger turn budgets.
- **Skills**: short how-tos the agent can read, to lift hard tasks from never to sometimes solved.
- **Large models on SageMaker HyperPod**: one node trains, one serves, over EFA; the sandboxes stay the same.

In plain English: the sandboxes keep up, rarely fail and grade honestly. What remains is RL work, making more tries differ so GRPO can learn, and scale.

Deep dive: “Results at a glance”, “Signal”, “What would make it learn”, “Next”

Key AI-infra results: the trainer waits for the model, not the sandbox

Figure 1 **Full-epoch run (4B, 13 Oct), medians: one training step, and one episode's sandbox time by substrate**

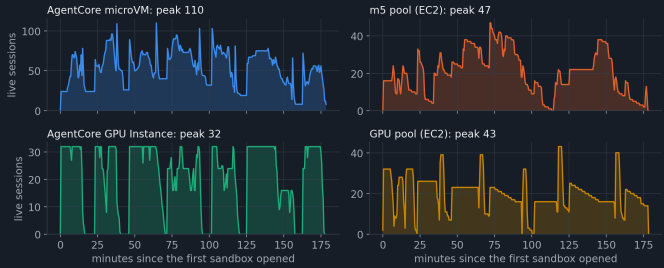


- **Step 88 s**: 55 s waiting for rollouts (model generation), 31 s training, 1.9 s weight sync.
- **Episode work 18–27 s** on every substrate: the substrate doesn't slow the agent.
- **Opens differ 100×** (0.9 s to 145 s); pre-open hides it: wait **0 s**.

Deep dive: "Time breakdown", "Sandbox latency"

Key AI-infra results: 192 live sandboxes across four substrates

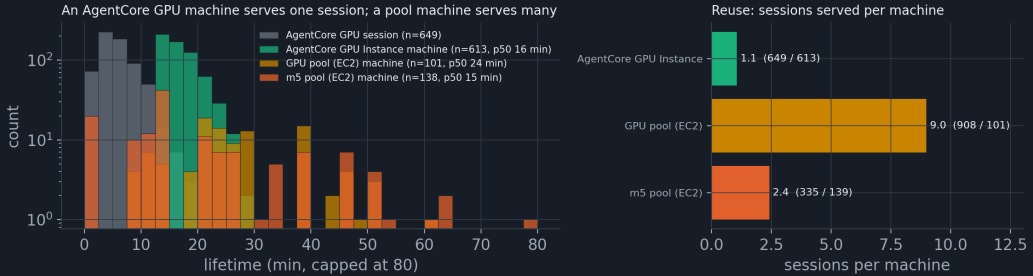
Figure 2 **Full-epoch run: live sandbox sessions per substrate**



- **192 live sandboxes** at peak: 110 microVM, 47 m5, 32 AgentCore GPU, 43 GPU pool.
- **32 episodes run at once**; the rest are pre-opened, ready for the next batches.
- **Dips** are eval and checkpoint pauses.
- **GPU capacity is the real limit**: g6 starved in all 4 AZs, so g5 did most of the work.

Key AI-infra results: GPU reuse sets the cost; errors are rare

Figure 3 **Full-epoch run (13 Oct): machine lifetimes and sessions served per machine**



- **AgentCore GPU**: one machine per session; it lives **16 min** for a **5.4 min** session.
- **EC2 pools reuse**: GPU pool **908** episodes on 101 machines (∼9 each); m5 2.4 each.
- **0.35 % infra errors** over all runs (69 in 19,606), mostly grading time-outs.

Getting started

What you need

- An AWS account in a Region with AgentCore Runtime (built and tested in us-west-2).
- A GPU host for training (here 8 GPUs), with the slime container.
- Quotas: AgentCore Runtimes and sessions, EC2 vCPU; and **GPU capacity**, ideally reserved.
- Your tasks in Terminal-Bench format: a Dockerfile, an instruction, hidden tests, a reference solution.

How it runs

1. Build the sandbox images and push them to Amazon ECR.
2. Register the Runtimes; every task's reference solution is checked where it will train.
3. Build the training data from the tasks that pass.
4. Train: a smoke run first ($\sim$20 min), then full runs.
5. Clean up: one script removes what the run created.

Plan for the first image build: hours for a few hundred tasks.

In plain English: bring tasks with hidden tests and a reference solution; the same pipeline routes, checks and trains them.

Deep dive: "Prerequisites", "Run it end to end", "Bring your own tasks"